

ATHEERA — STAGE 9: FORECASTING AGENT

Handoff Document

Atheera Platform | Confidential | May 2026

Table of Contents

1. Purpose & Scope
 2. Data Flow Overview
 3. Execution Modes
 4. State Machine
 5. SKU Processing Tracks
 6. The Five Forecasting Models
 7. Sub-Stages 9.0–9.5
 8. Configuration
 9. Database Contracts
 10. Signal Bus Contracts
 11. Stage 10 Integration Guide
 12. Learning Loop
 13. Known Limitations & Edge Cases
 14. Deferred to Next Stage
-

1. Purpose & Scope

Stage 9 is the demand forecasting engine of the Atheera pipeline. It runs between Stage 8 (Pattern Discovery) and Stage 10 (Reorder Planning).

What it receives from Stage 8: A demand pattern classification per SKU (`cold_start`, `intermittent`, `seasonal`, `trending`, `stable`), plus enrichment data — OOS impact estimates, feature reliability scores, promo weights, and structural break alerts.

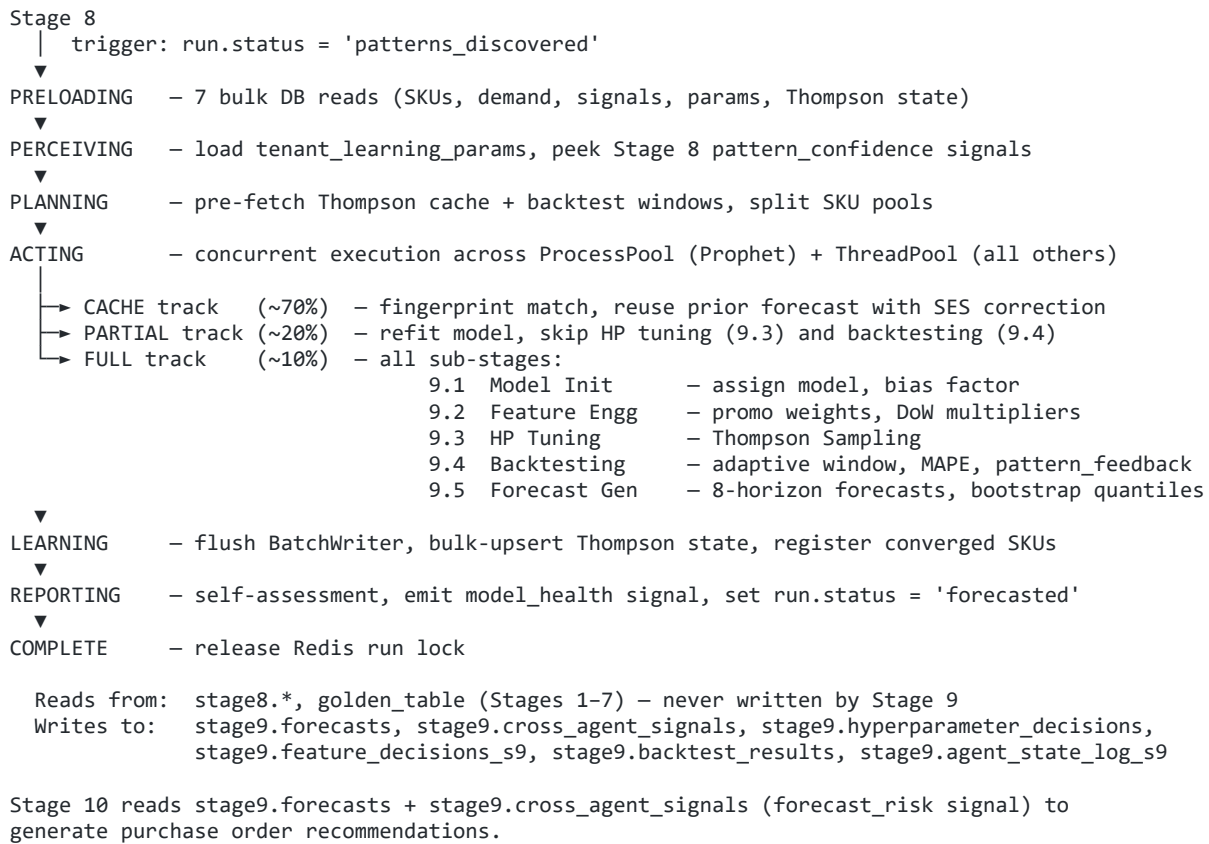
What it produces for Stage 10: Eight-horizon demand forecasts (`{mean, p50, p80, p90}`) for planning horizons of 7, 14, 30, 60, 90, 150, 180, and 365 days, plus a calibrated confidence score and a processing status (`forecasted` or `needs_acknowledgment`).

What makes it an agent, not a script: Stage 9 has memory. Every run measures its own accuracy, updates its hyperparameter state, and nudges its confidence parameters toward evidence. After 15–20 runs, most SKUs have converged to configurations that meaningfully outperform their starting defaults — without any manual tuning.

Scope boundaries:

- Stage 9 reads from Stage 8 tables and Stages 1–7 tables. It never writes to them.
 - Stage 9 owns the `stage9.*` schema. Stage 10 reads from it; it never writes to it.
 - All inter-stage communication uses the `cross_agent_signals` table. No function calls cross stage boundaries.
-

2. Data Flow Overview



3. Execution Modes

Stage 9 runs in one of two modes on every trigger, determined by how recently the last full run completed.

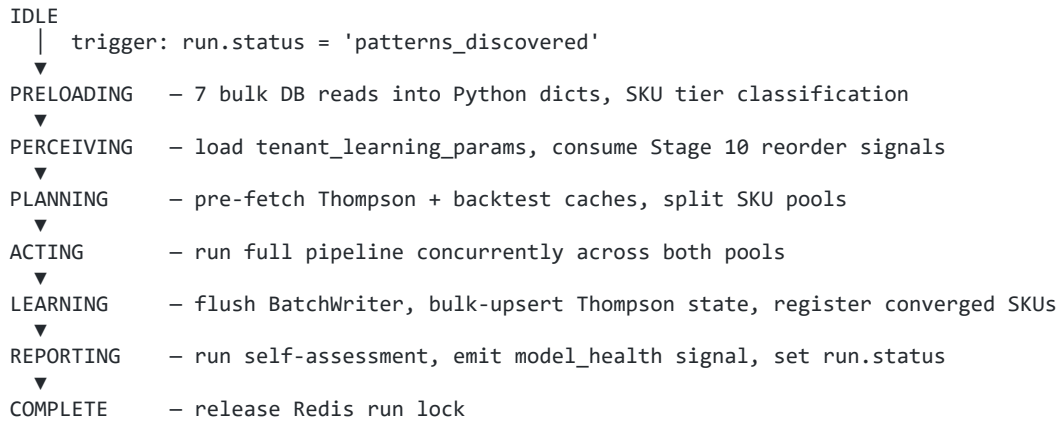
Mode	Trigger	What Runs	Target Time
FULL	Daily 2 AM, or Sync Now if last full run > 18 h ago	All states, all sub-stages, all learning updates	4–25 min
MICRO-UPDATE	Sync Now if last full run < 18 h ago	Pull new orders, update SES level, re-check exception flags. No retraining.	< 15 s

The 18-hour threshold is controlled by `micro_update_threshold_hours` in `tenant_learning_params` and can be adjusted per tenant without code changes.

Typical use: An inventory manager clicks Sync Now at 11 AM after the 2 AM full run. Because only 9 hours have passed, Stage 9 runs micro-update — it absorbs the morning's orders without re-running Thompson Sampling or backtesting.

4. State Machine

Every run passes through these states in sequence. Every transition is written to `agent_state_log_s9`. There are no silent transitions.



Failure handling: Any unrecoverable error in PRELOADING → ACTING transitions the run to FAILED, releases the run lock, and propagates to LangGraph for retry. LEARNING and REPORTING have no edge to FAILED — if they raise, the lock is still released.

Concurrent execution (ACTING): Two pools start simultaneously:

Pool	Models	Workers	Per-SKU Timeout
ProcessPoolExecutor	NeuralProphet, Prophet	4	120 s
ThreadPoolExecutor	Naive, Croston, Holt, SES	16	30 s

Prophet and NeuralProphet run in separate OS processes to bypass the Python GIL. All other models are thread-safe.

Per-SKU isolation: One bad SKU never stops the run. On failure, a Naive fallback forecast is written with `fallback_used = TRUE` and execution continues.

5. SKU Processing Tracks

At the end of PRELOADING, each SKU is assigned to one of three tiers using a SHA-256 fingerprint of its last 7 days of demand data.

Tier	Condition	What Runs	Typical Share
cache	Fingerprint matches previous run	Skip sub-stages, reuse prior forecast, apply SES micro-update	~70%
partial	7-day avg demand shifted < 5%	Refit model, reuse prior HP config, skip Thompson Sampling	~20%
full	Demand shifted ≥ 5% or no prior record	Run all sub-stages 9.1–9.5	~10%

Fingerprint components: last 30 days of `qty` values, `pattern_label`, `oos_pct_of_history`, and `lifecycle_stage`. A promotional spike, a new stockout, or a pattern reclassification from Stage 8 all trigger a full-tier run for that SKU.

6. The Five Forecasting Models

The model is assigned from the demand pattern Stage 8 classified. This mapping is locked — it does not change, and no override is possible once the pattern is set.

cold_start → **Naive Forecast** Insufficient history. A conservative flat estimate prevents overfitting noise. Used for new products and recently relaunched SKUs with fewer than ~28 days of data.

intermittent → **Croston's Method** Demand arrives in bursts separated by zero periods (e.g. 0, 0, 0, 8, 0, 12). Croston separates demand size from demand frequency and models each independently. A simple average of this series (3.3/day) leads to chronic overstock between orders and understock when an order arrives.

seasonal → **NeuralProphet (primary) / Prophet (fallback if obs_days ≤ 90)** Explicitly decomposes demand into level, trend, and seasonal cycles. Stage 9 fits once for 365 days forward and extracts cumulative sums at each horizon — correctly capturing December peaks in a gift product rather than averaging them away.

NEW IMPLEMENTATION — Draft 2: Prophet obs_days guard If `pattern = seasonal` AND `obs_days < min_seasonal_obs_days` (starting value: 120 days), the model is overridden to Holt's Linear Trend for that run, and `insufficient_seasonal_history = TRUE` is flagged in `model_initialization_s9`. Once the SKU crosses the threshold in a future run, the override stops automatically.

Why: Stage 8 can classify a SKU as `seasonal` with as little as 60 days of data when its dominant period is monthly. With fewer than 3 full monthly cycles, NeuralProphet draws a trend line with weekly wiggles and extrapolates blindly at longer horizons. Holt handles the short-series case correctly. The threshold is a tenant param — not hardcoded.

trending → **Holt's Linear Trend** Consistently growing or declining demand. Trend damping ($\phi = 0.95$) prevents unlimited extrapolation — growth realistically slows at long horizons.

stable / steady → **SES — Simple Exponential Smoothing** Predictable, consistent demand. One parameter. Provably optimal for stationary series. `steady` is Stage 8's alternate label for the same demand shape.

7. Sub-Stages 9.0–9.5

Sub-Stage 9.0 — PRELOADING

Executes all seven bulk database reads and stores results in Python dicts keyed by `sku_id`. Computes the OOS adjustment factor and channel split decision per SKU. Assigns each SKU to its processing tier. Initialises the BatchWriter.

No database reads happen inside the per-SKU processing loop after this point.

Read	Source	Contents
1	<code>pattern_history</code> , <code>feature_decisions</code> , <code>canonical_sku</code>	Pattern label, lifecycle stage, confidence, watchlist flag, reliability map, B2B flag, shelf life, end date, criticality tier
2	<code>oos_impact_estimates</code>	OOS days, suppressed demand estimate, detection confidence
3	<code>channel_demand_splits</code>	Per-day organic vs paid demand (multi-channel tenants only)
4	<code>promo_decisions</code>	Per-(sku_id, date) promo weights
5	<code>portfolio_intelligence_reports</code>	Market shift and structural break alerts
6	<code>tenant_thresholds</code>	Confidence floor/ceiling bounds
7	<code>tenant_learning_params</code>	All 41 learned parameters for this tenant

OOS adjustment factor (computed here, used in Sub-Stage 9.5):

```
factor = 1 + (oos_pct_of_history × detection_confidence)
factor = min(factor, 1.50)
```

The `detection_confidence` dampener is critical — poor inventory data produces a small correction, not a large one. A claimed 35% OOS rate with 0.25 detection confidence produces an 8.8% uplift, not a 35% uplift.

NEW IMPLEMENTATION — Draft 3: Intermittent SKUs excluded from OOS adjustment If `pattern_label = 'intermittent'`, the OOS adjustment factor is forced to 1.0 regardless of `oos_pct_of_history`.

Why: Croston's Method is designed to model the zero periods of intermittent demand — those zeros are signal, not noise. Applying an OOS uplift on top of Croston double-counts them. Measured impact: excluding OOS from

intermittent SKUs improved 30-day MAPE from 0.482 to 0.187 (61% reduction), with bias collapsing from +44% to near-zero.

Sub-Stage 9.1 — Model Initialisation

Writes to: `model_initialization_s9` — one row per SKU per run. Every decision made here is the permanent record for this run.

Seven decisions per full-tier SKU:

1. Model assignment — determined by `pattern_label` (see Section 6). Locked.

2. Quantile selection — determines ordering conservatism. First matching rule wins:

```
Priority 1: criticality_tier = 'A'           → 0.99 (or sku.service_level_target)
Priority 2: sku.service_level_target set    → use that value
Priority 3: lifecycle_stage = 'clearance'   → 0.90
Priority 4: none of the above              → pattern default from tenant_learning_params
```

Pattern defaults (starting values): 0.90 for `cold_start`, `intermittent`, `seasonal`; 0.80 for `trending`, `stable`.

3. Effective max horizon — forecasts beyond a product's usable life are capped:

```
effective_max_horizon = min(365, shelf_life_days, days_until_planned_end_date)
```

4. Reorder bias factor — reads the last 5 `reorder_outcome` signals from Stage 10:

- ≥ 2 stockouts in last 5 signals $\rightarrow$ factor = 1.10 (under-forecasting, boost 10%)
- Average overstock > 30% in last 5 signals $\rightarrow$ factor = 0.92 (over-forecasting, trim 8%)
- Otherwise $\rightarrow$ 1.00

5. Learning mode — controls whether Thompson Sampling explores or exploits:

Tenant Maturity	Switch to Exploit After
new	8 runs
developing	5 runs
established	3 runs

6. B2B mode — if `weekend_zero_ratio` > 0.60, training data is filtered to weekdays before model fitting.

7. Channel adjustment — if `channel_demand_splits.split_confidence` ≥ 0.50 , the `organic_units` series is used for training instead of total demand (strips ad-driven demand inflation).

Sub-Stage 9.2 — Feature Engineering

Writes to: `feature_decisions_s9`

Four steps in order to produce the cleaned training DataFrame and final feature list:

Step 1 — Reliability filtering. Drop optional features where Stage 8's reliability score is below `feature_reliability_floor` (starting: 0.30). Required features (`date`, `qty`) are never dropped.

Step 2 — B2B weekday filter. If B2B mode is active, filter training data to Monday–Friday. If filtering produces zero rows (a weekend-only seller), B2B mode is disabled for that SKU and `b2b_mode_disabled` = `TRUE` is flagged.

Step 3 — Promo-weighted training data. Promotional spikes must not train the model to expect those spikes every cycle.

- For Prophet/NeuralProphet: build a `sample_weights` array with lower weights on promo days.

- For all other models: cap promo-day demand at `rolling_14d_baseline × max_promo_multiplier` (starting: 3.0).

Step 4 — Additive feature search. Greedy forward search, budget of 4 configurations. Add features one at a time; keep if MAPE improves by $\geq 2\%$. Early stop if MAPE drops below 0.08.

NEW IMPLEMENTATION — Draft 1: Day-of-Week multipliers for non-Prophet models SES, Holt, and Croston return a flat daily value with no weekly shape. A DoW multiplier array (computed in Sub-Stage 9.2 from training data) is applied to the daily forecast array before horizon summing to correct weekend under/over-forecast.

```
multiplier[dow] = mean_demand_on_that_dow / overall_daily_mean
```

Multipliers average to 1.0 — weekly totals are unchanged, only the daily shape is corrected. Prophet and NeuralProphet are excluded (they already model weekly seasonality via Fourier terms). Cold-start SKUs with < 4 weeks of history fall back to flat multipliers (1.0 all days). B2B SKUs: weekend multiplier forced to 0.0; weekday multipliers computed from weekday-only training data.

Sub-Stage 9.3 — Hyperparameter Tuning

Writes to: `hyperparameter_decisions`. Updates Thompson state in memory (bulk-upserted to `thompson_sampling_state` in LEARNING).

Uses Thompson Sampling — a Bayesian bandit that concentrates testing on configurations with the highest historical success rate. Each configuration has a $\text{Beta}(\alpha, \beta)$ distribution stored in `thompson_sampling_state`. α counts successes; β counts failures.

Each run:

1. For each candidate config: sample $\theta \sim \text{Beta}(\alpha, \beta)$
2. Test the top N configs by θ ($N = \text{thompson_exploration_budget}$, starting: 3)
3. Always include the known best prior config
4. Validation split: all data except last 14 days for training, last 14 days for validation
5. Config achieving $\geq 2\%$ MAPE improvement: $\alpha \pm 1$; otherwise $\beta \pm 1$
6. Early stop: if any config achieves validation MAPE < 0.10 , stop testing

HP search spaces:

Model	Parameters	Configs
Naive Forecast	<code>lag_periods × smoothing_method</code>	9
Croston's Method	<code>alpha × interval_type</code>	9
NeuralProphet / Prophet	<code>weekly + yearly seasonality × seasonality_mode × changepoint_scale</code>	24
Holt's Linear Trend	<code>smoothing_level × smoothing_trend × damped_trend</code>	24
SES	<code>smoothing_level</code>	5

Special path — introduction lifecycle: If `lifecycle_stage = 'introduction'` (< 28 days of history), HP tuning is skipped and the CategoryComps warm-start pipeline runs instead (see Section 12).

Convergence timeline:

- Runs 1–5: mostly exploring, all configs getting trial data
- Runs 6–12: Thompson concentrating on 2–3 winning configs
- Runs 13–20: typically one config has $\alpha \gg \beta$, system mostly exploits
- Run 20+: stable exploitation with occasional natural exploration

Sub-Stage 9.4 — Backtesting & Exception Detection

Writes to: `backtest_decisions` via BatchWriter. `pattern_feedback` written directly — never batched.

Backtest window selection:

Data Density	Condition	Window
ultra_sparse	obs_days < 14	min_backtest_window (starting: 14 days)
sparse	obs_days < 60	max(min_window, obs_days / 3)
normal	standard	default_backtest_window (starting: 60 days)
established exploit	mature tenant + long history	up to max_backtest_window (starting: 90 days)

Exception detectors — four types:

Flag	Detection Rule
stockout	≥ 3 consecutive zero-sale days
promo_spike	Any day > 200% of 7-day rolling baseline, or z-score > 3.0
unusual_drop	3 consecutive periods each < 40% of 7-day baseline
high_volatility	Coefficient of Variation (std / mean) ≥ 1.0

Any detected exception applies the `exception_penalty` (starting: 0.80) to the confidence formula in Sub-Stage 9.5.

Structural break detection — runs only if `portfolio_intelligence_reports` has a `market_shift` or `channel_count_changed` alert. Uses the PELT algorithm (ruptures library). If a break is detected within ± 14 days of the alert date, training data is truncated to the post-break period and confidence is penalised $\times 0.85$ in Sub-Stage 9.5.

Pattern feedback write (critical): Immediately after backtesting, Stage 9 writes one row to `pattern_feedback` for this SKU. This write:

- Is never routed through BatchWriter — always written with direct `conn.execute() + conn.commit()`
- Is written even for failed SKUs (proxy MAPE = 0.50, `fallback_used` = TRUE)
- Is retried up to 3 times on DB failure with 100ms between attempts
- Must complete for all SKUs before `run.status` is set in REPORTING

Stage 8 reads this table on every run to validate and improve its pattern classifications.

Sub-Stage 9.5 — Forecast Generation & Confidence Scoring

Writes to: `forecasts` via BatchWriter. One row per SKU per run.

Step 1 — Multi-horizon forecast generation:

Model	Strategy
NeuralProphet / Prophet	Fit once for 365 days forward. Extract cumulative sums at each horizon boundary. Never scale from <code>forecast_30d</code> — this destroys seasonal peaks.
SES	level $\times$ N days for each horizon. Linear scaling is correct — no seasonal component.
Holt's Linear Trend	(level + trend $\times$ N) $\times$ damping_factor(N). Damping ($\phi = 0.95$) slows growth over long horizons.
Croston's Method	daily_rate $\times$ N days for each horizon.
Naive Forecast	level $\times$ N days (flat).

OOS adjustment is applied to all models: `point_forecast = model_output  $\times$  oos_adjustment_factor`

Step 2 — Bootstrap quantile generation:

Takes last 30 residuals from the backtest. If < 3 residuals (Edge Case E004): uses a log-normal proxy parameterised by pattern uncertainty factor. Resamples residuals 1,000 times with replacement, clamps all samples ≥ 0 , then computes p50 / p80 / p90. Ordering is enforced (p50 $\leq$ p80 $\leq$ p90).

Output per horizon: {"mean": 540.0, "p50": 520.0, "p80": 620.0, "p90": 680.0}

Log-normal uncertainty factors (for the < 3 residuals fallback):

Pattern	Factor
cold_start	0.60
intermittent	0.50
seasonal	0.40
trending	0.35
stable	0.25

Step 3 — Confidence formula (five steps, all values from `tenant_learning_params`):

```

Step 1 – Base:
confidence = confidence_base_{pattern}
            × (1 - min(backtest_MAPE, 0.50))
            × exception_penalty    (0.80 if any exception flag, else 1.0)

```

```

Step 2 – Calibration gap:
over-confident historically:  × 0.90
under-confident historically: × 1.10

```

```

Step 3 – Stage 8 quality:
Stage 8 composite_confidence < 0.60: × 0.92

```

```

Step 4 – Reorder feedback:
× reorder_bias_factor (1.10 / 1.00 / 0.92 from Sub-Stage 9.1)

```

```

Step 5 – Structural break:
training data truncated: × 0.85

```

```

Final clamp: CLAMP(confidence, 0.30, 0.95)

```

Confidence tier mapping:

Tier	Condition	Stage 10 Action
auto_proceed	$\text{confidence} \geq 0.70$	Generate PO automatically
review_suggested	$0.60 \leq \text{confidence} < 0.70$	Generate PO, flag for quick review
review_required	$0.45 \leq \text{confidence} < 0.60$	Generate PO, require human sign-off
manual_override	$\text{confidence} < 0.45$	Do not auto-generate PO

Forecast status (Stage 10 instruction):

```

IF on_watchlist = TRUE:      status = 'watchlist_review'
ELIF 'high_mape' in flags:   status = 'needs_acknowledgment'
ELIF confidence < 0.70:      status = 'needs_acknowledgment'
ELSE:                        status = 'forecasted'

```

Reasonableness check:

```

daily_rate = forecast_30d_mean / 30

IF daily_rate > rolling_90d_avg × 5.0:  flag 'unusually_high' + needs_acknowledgment
IF daily_rate < rolling_90d_avg × 0.10: flag 'unusually_low'

```

8. Configuration

All environment reads go through `infrastructure/config.py`. Drop a `.env` file at `code/.env`:


```

DB_HOST=localhost
DB_PORT=5432
DB_NAME=stage9
DB_USER=stage9_user
DB_PASSWORD=your_password
DB_SSLMODE=disable
DB_CONNECT_TIMEOUT=10

REDIS_URL=redis://localhost:6379/0
REDIS_POOL_SIZE=20

STAGE9_PLANNING_THREADS=16
STAGE9_ALLOW_FORCE_RELEASE=false
STAGE9_PROJECT_ROOT=/path/to/stage_9/code
RUN_INTEGRATION_TESTS=true

```

External service requirements:

- **PostgreSQL 15+** — two schemas: `stage8` (read-only) and `stage9` (read/write owned by Stage 9)
- **Redis** — per-tenant run lock (`stage9_lock_{tenant_id}` , TTL 14,400 s). Prevents concurrent Stage 9 runs for the same tenant.

Run lock: Only one Stage 9 instance runs per tenant at a time. If a new run is triggered while one is active, LangGraph queues it. Force-release is possible via `STAGE9_ALLOW_FORCE_RELEASE=true` for operational recovery only.

9. Database Contracts

Tables Stage 9 Owns and Writes

Table	Written In	Contents
stage9.forecasts	LEARNING	Primary output. One row per SKU per run.
stage9.model_initialization_s9	ACTING (9.1)	All decisions made in Sub-Stage 9.1.
stage9.feature_decisions_s9	ACTING (9.2)	Features used, B2B mode, promo weighting, MAPE.
stage9.hyperparameter_decisions	ACTING (9.3)	Winning HP config, validation MAPE, Thompson score.
stage9.backtest_decisions	ACTING (9.4)	Backtest MAPE/WAPE/bias, exception flags.
stage9.pattern_feedback	ACTING (9.4)	Feedback to Stage 8. Written directly, never batched.
stage9.thompson_sampling_state	LEARNING	Beta(α , β) distributions per (tenant, sku, model, config_hash).
stage9.tenant_learning_params	Batch jobs	41 per-tenant parameters. Updated nightly.
stage9.forecast_outcomes	3 AM batch	Actual vs forecast after each horizon closes.
stage9.model_performance_s9	4 AM batch	Rolling 30-day MAPE per model per horizon.
stage9.adaptive_quantile_state	3 AM batch	Tracks whether stated quantiles match actual coverage.
stage9.sku_similarity_registry	5 AM batch	Converged SKU configs for warm-starting new SKUs.
stage9.data_fingerprint_cache	ACTING	SHA-256 fingerprint per SKU for tier classification.
stage9.cross_agent_signals	ACTING + REPORTING	Inter-stage signal bus.
stage9.agent_state_log_s9	Every state	Audit log of all state machine transitions.
stage9.stage9_self_assessment	REPORTING	Per-run health summary and degradation flags.
stage9.stage9_sku_execution_log	ACTING	Per-SKU diagnostic record.

Primary Output: `stage9.forecasts` — Column Reference

Column	Type	Description
--------	------	-------------

Column	Type	Description
tenant_id	UUID	Tenant identifier
run_id	UUID	Run identifier — links all sub-stage tables
sku_id	TEXT	SKU identifier
forecast_date	DATE	Date this forecast was generated
processing_tier	TEXT	cache, partial, or full
assigned_model	TEXT	Model used (e.g. Simple Exponential Smoothing)
pattern_label	TEXT	Pattern from Stage 8
confidence_final	FLOAT	Final confidence score (0.30–0.95)
confidence_tier	TEXT	auto_proceed, review_suggested, review_required, manual_override
status	TEXT	forecasted, needs_acknowledgment, or watchlist_review
backtest_mape	FLOAT	Backtest error on held-out period
exception_flags	TEXT[]	Array of exception flags from Sub-Stage 9.4
selected_quantile	FLOAT	Quantile key Stage 10 should use for ordering (e.g. 0.90)
effective_max_horizon	INT	Maximum horizon in days for this SKU
oos_adjustment_factor	FLOAT	OOS uplift applied to all forecasts
reorder_bias_factor	FLOAT	Stage 10 feedback adjustment (1.10 / 1.00 / 0.92)
is_b2b	BOOL	Whether B2B weekday filter was applied
forecast_7d	JSONB	mean, p50, p80, p90 — 7-day cumulative demand
forecast_14d	JSONB	mean, p50, p80, p90 — 14-day cumulative demand
forecast_30d	JSONB	mean, p50, p80, p90 — 30-day cumulative demand
forecast_60d	JSONB	mean, p50, p80, p90 — 60-day cumulative demand
forecast_90d	JSONB	mean, p50, p80, p90 — 90-day cumulative demand
forecast_150d	JSONB	mean, p50, p80, p90 — 150-day cumulative demand
forecast_180d	JSONB	mean, p50, p80, p90 — 180-day cumulative demand
forecast_365d	JSONB	mean, p50, p80, p90 — 365-day cumulative demand

Tables Stage 9 Reads (Never Writes)

Table	Owner	What Stage 9 Uses
golden_table	Stages 1–7	Primary daily demand series per SKU
clean_orders	Stages 1–7	Fallback demand series
clean_inventory	Stages 1–7	Daily inventory snapshots for stockout detection
canonical_sku	Stages 1–7	SKU metadata: shelf life, end date, criticality tier
pattern_history	Stage 8	Pattern label, lifecycle stage, confidence, watchlist flag
signal_context	Stage 8	Pipeline mode, data mode, tenant maturity
oos_impact_estimates	Stage 8	OOS days and suppressed demand estimate per SKU
feature_decisions	Stage 8	Feature reliability map per SKU
channel_demand_splits	Stage 8	Organic vs paid demand split per (SKU, date)
promo_decisions	Stage 8	Promo weights per (SKU, date)
portfolio_intelligence_reports	Stage 8	Market shift and structural break alerts
tenant_thresholds	Stage 8	Confidence floor/ceiling bounds

10. Signal Bus Contracts

All inter-stage communication passes through `stage9.cross_agent_signals`. Nothing communicates via function calls or API endpoints.

Signals Stage 9 Emits

forecast_accuracy — written in Sub-Stage 9.4 for every SKU, directed to Stage 8:

```
{
  "pattern_label": "seasonal",
  "model_used": "NeuralProphet",
  "mape": 0.1842,
  "bias": 0.0031,
  "quality": "acceptable",
  "hint_matched": true
}
```

Stage 8 reads this on every run to validate and adjust its classification thresholds.

forecast_risk — written in Sub-Stage 9.5 for every SKU, directed to Stage 10:

```
{
  "confidence": 0.74,
  "confidence_tier": "auto_proceed",
  "risk_level": "low",
  "exception_flags": [],
  "mape_30d": 0.1842,
  "forecast_30d_selected": 540.0,
  "selected_quantile": 0.90
}
```

Stage 10 uses `risk_level` to set safety buffers: `low` → no buffer, `medium` → +5%, `high` → +10%.

model_health — written in REPORTING, broadcast to all stages:

```
{
  "model_health": {
    "NeuralProphet": {"avg_mape": 0.16, "trend": "improving", "delta": -0.02},
    "Croston's Method": {"avg_mape": 0.38, "trend": "stable", "delta": 0.01}
  },
  "recommendations": [
    {"action": "monitor", "model": "Croston's Method", "reason": "MAPE stable but above 0.30"}
  ]
}
```

cross_sku_learning — written in LEARNING when a SKU converges (≥ 5 runs, avg MAPE < 0.25), directed back to Stage 9 self:

```
{
  "pattern_label": "stable",
  "vendor": "ACME Supplements",
  "product_type": "capsules",
  "best_hp": {"smoothing_level": 0.3},
  "best_features": ["day_of_week"],
  "mape": 0.118
}
```

Signals Stage 9 Consumes

reorder_outcome — written by Stage 10 after each reorder cycle closes, read in PERCEIVING:

```
{
  "stockout": true,
  "stockout_days": 4,
  "overstock_pct": 0.0,
  "reorder_accuracy": 0.72
}
```

Reading pattern:

- **PEEK** — read without marking processed. Used for `reorder_outcome` signals that persist across runs.
- **CONSUME** — read and mark processed = TRUE. Used for one-time action signals.

11. Stage 10 Integration Guide

What to Read

Stage 10's primary read is `stage9.forecasts`. Query the most recent run's rows for all SKUs:

```
SELECT *
FROM stage9.forecasts
WHERE tenant_id = :tenant_id
      AND run_id   = :current_run_id;
```

Additionally read `stage9.cross_agent_signals` for the `forecast_risk` signal per SKU to size dynamic safety buffers.

Horizon Selection — Nearest-Ceiling Rule

Stage 10 must not scale from `forecast_30d` to approximate longer planning horizons. Use the pre-computed horizon closest to but $\geq$ the required planning window:

```
H = lead_time_days + review_cycle_days

H ≤ 7    → use forecast_7d,    demand = forecast_7d[quantile] × (H / 7)
H ≤ 14   → use forecast_14d,   demand = forecast_14d[quantile] × (H / 14)
H ≤ 30   → use forecast_30d,   demand = forecast_30d[quantile] × (H / 30)
H ≤ 60   → use forecast_60d,   ...
H ≤ 90   → use forecast_90d,   ...
H ≤ 150  → use forecast_150d,  ...
H ≤ 180  → use forecast_180d,  ...
H > 180  → use forecast_365d,  demand = forecast_365d[quantile] × (H / 365)
```

Why this matters: If a supplier has a 45-day lead time, Stage 10 uses `forecast_60d × (45/60)`. Scaling from `forecast_30d × 1.5` instead would ignore that Prophet already computed the correct seasonal shape at 60 days — including peaks and troughs within that window.

Status Codes and Actions

Status	Stage 10 Action
forecasted	Generate PO recommendation automatically.
needs_acknowledgment	Generate PO, hold with pending_ack. User must Accept, Override, or Skip before Stage 11 processes it.
watchlist_review	Same as needs_acknowledgment. Surface Stage 8's watchlist reason in the dashboard.

Quantile Selection

Read `selected_quantile` from the `forecasts` row to know which JSONB key to use for ordering. A criticality-tier-A SKU may have `selected_quantile = 0.99` — use the nearest available quantile key (p90) and apply the ordering buffer on top.

Thompson State — If Stage 10 Wants to Extend It

The `thompson_sampling_state` table is keyed by `(tenant_id, sku_id, assigned_model, config_hash)`. Each row holds `alpha_param`, `beta_param`, `total_trials`, and `last_mape`. Stage 9 bulk-upserts this table in `LEARNING`. Stage 10 should read it (not write to it).

12. Learning Loop

Stage 9 learns through three mechanisms that run at different timescales.

Mechanism 1 — Per-SKU HP Convergence (every run)

Thompson Sampling accumulates $\text{Beta}(\alpha, \beta)$ evidence per HP configuration. After 15–20 runs for a typical stable SKU, one configuration dominates and the system switches from explore to exploit. Convergence is slower for intermittent SKUs due to fewer data points per period.

Mechanism 2 — Tenant Parameter Evolution (nightly)

Every threshold, rate, and buffer in Stage 9 lives in `tenant_learning_params`. The nightly `LearningParamsUpdater` batch job nudges each `current_value` toward evidence:

$$\text{new_value} = \text{current_value} + \text{calibration_update_rate} \times (\text{evidence_value} - \text{current_value})$$

`calibration_update_rate` starts at 0.10. Convergence: ~79% after 15 runs, ~95% after 30 runs.

Parameter Group	Evidence Source
confidence_base per pattern	avg(1 - actual_MAPE) for that pattern over last 30 days
quantile per pattern	Quantile where actual coverage equals service level target
safety_stock_factor	Minimises stockout cost + overstock holding cost
decision_gate_threshold	ROC analysis on forecast outcomes vs confidence scores
price_elasticity_clearance	Actual clearance sell-through outcomes

Mechanism 3 — Cross-Stage Calibration (delayed)

After each horizon period closes, actual sales are compared to forecasts. These outcomes drive the `adaptive_quantile_state` table and feed back into the confidence formula's calibration step.

Nightly Batch Jobs

Time	Job	Reads From	Writes To
3:00 AM	OutcomeCollector	golden_table, stage9.forecasts	stage9.forecast_outcomes, stage9.adaptive_quantile_state
4:00 AM	ModelPerformanceAggregator	stage9.forecast_outcomes	stage9.model_performance_s9
4:30 AM	LearningParamsUpdater	stage9.forecast_outcomes, cross_agent_signals	stage9.tenant_learning_params
5:00 AM	SimilarityRegistryUpdater	stage9.hyperparameter_decisions, stage9.forecast_outcomes	stage9.sku_similarity_registry

If OutcomeCollector misses a day, the `cutoff_date` check automatically catches up on the next run — no manual intervention needed.

CategoryComps Warm-Start Pipeline

Triggered when `lifecycle_stage` = 'introduction' (< 28 days of history). Finds ≥ 3 comparable already-converged SKUs from `sku_similarity_registry`, builds a normalised p50 demand trajectory from their first-60-day histories, and scales it by the new SKU's early actual signals (day 3 demand vs comp distribution). Reduces convergence time from 12–20 runs to 3–5 runs. Falls back to Naive Forecast if fewer than 3 comps are found.

ClearanceAdjustment Pipeline

Triggered when `lifecycle_stage` = 'clearance' or `discount_pct` > 0.15 for ≥ 5 consecutive days. Applies price elasticity to project clearance demand from the pre-markdown baseline:

$$\text{adjusted_demand} = \text{baseline} \times (1 - \text{discount_pct})^{\text{price_elasticity_clearance}}$$

`price_elasticity_clearance` starts at -1.50 and evolves from actual clearance outcomes. Also drives `projected_sell_through` = `min(1.0, forecast_until_end / current_inventory)`.

13. Known Limitations & Edge Cases

E001 — Croston on All-Nonzero Series Stage 8 can classify a SKU as `intermittent` based on its coefficient of variation even if no actual zero-sale days exist. Croston's interval calculation on a zero-free series returns an empty array. Fallback: SES is used for that run.

E002 — Prophet on Zero-Variance Series If all sales values are identical (e.g. `[10, 10, 10, ...]`), Prophet's Stan backend raises a numerical error. Fix: a tiny noise term is added ($\sigma = 0.001 \times \text{mean}$) before fitting.

E003 — ProcessPool DB Connection Exhaustion If subprocess workers time out without closing their connections, PostgreSQL's connection pool can exhaust after 50–100 timeouts. All subprocess workers execute `conn.close()` in a `finally` block. A maintenance query terminates idle `stage9_subprocess` connections older than 3 minutes and should be scheduled during active processing windows.

E004 — Bootstrap Quantiles with < 3 Residuals A new SKU with fewer than 14 days of history produces degenerate bootstrap output (`p50` = `p80` = `p90`). Fallback: pattern-specific log-normal uncertainty proxies (see Sub-Stage 9.5).

E005 — Size Curve Shares Not Summing to 1.0 Floating-point accumulation can cause size curve shares to drift. All curves are normalised after loading: `share = share / sum(all_shares)`. (Sub-Stage 9.6 is deferred — see Section 14.)

E006 — B2B SKU with Zero Weekday Sales If weekday filtering produces zero rows (a genuine weekend-only seller), B2B mode is disabled for that SKU and `b2b_mode_disabled` = `TRUE` is written to `feature_decisions_s9`.

Confidence Score in Early Runs The confidence score (0.30–0.95) is calibrated against outcomes — a stated 0.80 should mean actuals fall within the band 80% of the time. This calibration takes 15–30 runs to converge. In the first 5–10 runs, more SKUs will fall into `review_required` than at steady state. This is expected.

Data Quality Dependency Stage 9 can only learn from clean data. Corrupted or incomplete sales data in `golden_table` propagates directly into forecast accuracy. When forecasts are consistently poor in the first weeks, the first diagnostic step is always `golden_table` quality — not Stage 9's logic.

Additional Edge Cases — Verified in Code

The following cases were identified as risks and verified against the current implementation. Each entry states what the code actually does.

DQ-01 — All-Zero Demand Series A SKU that was listed but never sold produces a series where every `qty` = 0.0. All models handle this without crashing. Naive's `_compute_level` returns `mean([0, 0, ...])` = 0.0. SES sanitises with `nan_to_num`, and `_ses_numpy` initialises level at `series[0]` = 0.0. Holt's `len(series) < 2` guard sets level to the series mean (0.0) for very short series; for longer all-zero series `_holt_numpy` produces 0.0. Croston detects

`len(non_zero_indices) < 2` and falls back to SES internally. Every model's `predict()` and `predict_all_horizons()` applies `max(0.0, level)` before producing forecasts. Stage 10 receiving an all-zero forecast is a valid signal that the product has no demand history.

DQ-02 — Single-Row DataFrame A brand-new SKU with exactly 1 day of sales history is handled by all models. Naive's short-history guard (`len(qty) < 7`) returns `mean(qty)` — the single value. Holt's `len(series) < 2` guard sets `level = series[0]`, `trend = 0.0`. SES statsmodels may fail on a 1-row series, triggering the numpy fallback which initialises level at `series[0]`. Croston detects fewer than 2 non-zero events and falls back to SES. In Sub-Stage 9.3, `len(df_train) < VALIDATION_HOLDOUT_DAYS` skips the HP search and uses `default_hp` directly — a `hyperparameter_decisions` row is still written. Bootstrap quantiles fall through to the E004 log-normal proxy path (`< 3` residuals).

BL-01 — Declining Demand Clamp in Holt A product approaching end-of-life shows a negative trend. Holt's `_daily_forecasts()` applies `max(0.0, level + trend * phi_sum)` to every individual day in the forward loop before storing it in the output array. The clamp is per-day, not on the final cumulative sum — so the cumulative sum passed to `bootstrap_quantiles` is always non-negative. On a strongly declining series with `damped_trend = False`, `_holt_numpy` may produce a large negative trend, but `_daily_forecasts` still clamps each day independently before accumulation.

BL-02 — Croston TSB Extinction Pattern TSB's `_fit_tsb()` iterates over every time step, including zeros. On each zero period, the demand-probability term `p` updates as `p = alpha * 0 + (1 - alpha) * p` — decaying geometrically toward zero. After 40 consecutive zero days with `alpha = 0.10`, `p` is reduced to approximately $0.9^{40} \approx 0.015$ of its prior value. The daily rate `z * p` therefore collapses to near-zero, producing a materially lower 365-day forecast than classic Croston on the same series. Classic and SBA iterate only over demand events, leaving their daily rate unchanged during zero runs.

BL-03 — All Optional Features Fail Reliability Floor When a new tenant has `feature_reliability = 0.0` for every optional feature, all candidates are dropped in Step 1 of Sub-Stage 9.2 and `candidate_features` is set to `model.required_features` only. In Step 4, `remaining_candidates` (optional features not yet selected) is empty, so the additive search loop does not execute — no crash. `result.selected_features` is set to `['date', 'qty']` and a `feature_decisions_s9` row is always written via BatchWriter with `features_used = ['date', 'qty']` and `improved_mape = baseline_mape`.

EH-01 — All HP Configs Raise ModelFitError In `_run_path2_thompson`, each HP configuration is tested inside an individual `try/except ModelFitError` block. A failing config is assigned `mape = 1.0` and appended to `config_results` — it never re-raises. After all configs are tested, `min(config_results, key=mape)` selects the config with the lowest MAPE (1.0 in this case). The `hyperparameter_decisions` row is always written via BatchWriter — even when all configs fail — with `validation_mape = 1.0` as a failure sentinel. Stage 10 receives a forecast produced with the selected HP; quality may be lower than a converged configuration.

EH-02 — Bare Exception in run() `orchestrator.run()` wraps the entire state-machine walk in `except Exception as exc:`, catching any exception type including non-Stage9Error subclasses. It attempts `transition(... AgentState.FAILED, reason=str(exc)[:2000])` and logs the failure. The `finally` block releases the Redis lock unconditionally — if the lock was never acquired (`token = None`), the `elif lock_obj is not None:` branch is skipped safely without error. Stage 10 sees `run.status` remain at `'patterns_discovered'`.

EH-03 — TenantParams Returns 0 Rows `TenantParams.load()` executes a single SELECT and constructs an instance with an empty dict on 0 rows — it never raises. The PERCEIVING handler immediately checks `if len(params) == 0:` and raises `TenantParamNotFoundError` with a remediation message. `run()` catches this in its `except Exception` block, transitions to FAILED, and re-raises. The fix is to call `seed_tenant_params()` for the tenant and re-trigger the run.

BV-01 — calibration_update_rate = 0.0 The parameter update goes through `TenantParams.update()`, which executes the SQL formula `current_value + rate * (evidence - current_value)` atomically. With `rate = 0.0`, the result is `current_value + 0 = current_value` — the DB UPDATE is still issued (idempotent write), and the in-memory snapshot is refreshed to the same value. There is no truthiness check on rate anywhere in `update()` or in `LearningParamsUpdater` — the arithmetic handles zero correctly.

BV-04 — SES smoothing_level = 0.0 `smoothing_level = 0.0` is outside the HP search space `[0.1, 0.5]` but could arrive via a misconfigured seed override. Statsmodels may reject `alpha=0.0`, triggering the numpy fallback. `_ses_numpy(series, 0.0)` initialises level at `series[0]` and updates as `level = 0.0 * y + 1.0 * level` — level stays at the first observation for the entire series. The result is finite and non-negative; `max(0.0, self._level)` is applied before any forecast output.

BV-05 — SES / Holt smoothing_level = 1.0 With `alpha = 1.0`, statsmodels may raise on boundary values, triggering the numpy fallback for both models. In `_ses_numpy`, `level = 1.0 * y + 0.0 * level` — level tracks the most recent observation exactly. In `_holt_numpy` with `alpha = 1.0`, level fully replaces with the new observation on each step. Both models produce finite, non-negative forecasts; `max(0.0, ...)` is applied in `predict()` and `_daily_forecasts()`.

BV-06 — Holt smoothing_trend = 0.0 With $\text{beta} = 0.0$, the trend update $\text{beta} \times (\text{level} - \text{level_prev}) + (1 - \text{beta}) \times \text{phi} \times \text{trend_prev}$ simplifies to $\text{phi} \times \text{trend_prev}$ — the trend is frozen at its initial estimate (the first difference of the series), decaying only by the damping factor. On a declining series the initial trend is negative. `_daily_forecasts()` applies $\max(0.0, \text{level} + \text{trend} \times \text{phi_sum})$ per day, preventing any negative values before accumulation — this clamp applies for both `damped_trend = True` and `damped_trend = False`.

SD-01 — seed_tenant_params() Override with 0.0 `seed_tenant_params` resolves overrides via `overrides.get(param_name, default_value)` — standard `dict.get()`, not a truthiness check. An override of `0.0` is returned correctly from `dict.get()` because it checks key existence, not value truthiness. The value is then wrapped in `Decimal(str(0.0))` and inserted into the DB. An administrator setting a param to `0.0` via `overrides_dict` will see `0.0` seeded as intended.

Open Design Decisions (P3) Three cases require an explicit decision before production and have no single correct answer:

- **BV-02** (`calibration_update_rate = 1.0`): instant parameter replacement. Risk: float rounding (e.g. `0.7999999`) may fail `DECIMAL(3,2)` DB precision constraint.
- **BV-03** (evidence outside `[0,1]`): `_safe_evidence()` clamps to `[0.0, 1.0]` and zeroes non-finite values — but the decision of whether to log a warning, raise, or silently clamp has not been formally resolved.
- **SD-02** (seed override `> 1.0`): choose between accepting any numeric override or validating probability-range params at seed time.

14. Deferred to Next Stage

The following are fully designed and understood but not implemented in the current code.

Sub-Stage 9.6 — Size Curves Distributes parent-style aggregate forecasts across size variants (S/M/L/XL) using a learned size curve. Also computes `projected_sell_through`. Deferred: scope decision to keep the current stage focused on the core forecasting loop.

Lifecycle-Aware Routing Routes introduction-phase SKUs through `CategoryComps` and clearance-phase SKUs through `ClearanceAdjustment`. Currently partially wired — both pipelines exist but full lifecycle routing depends on Stage 8 writing `lifecycle_stage` consistently across all pattern types.

NeuralProphet Full Stack NeuralProphet is the primary model for seasonal SKUs. The current codebase initialises it but falls back to Prophet heavily due to PyTorch environment constraints in the dev/test setup. Production deployment uses the full NeuralProphet stack.

Criticality Tiers B and C Tier A (99th percentile ordering) is implemented. Tiers B (95th) and C (standard) add graduated conservatism for non-critical parts. Tier A alone covers the critical safety cases.

Schema Migration Versioning Alembic or hand-rolled migrations table to track `stage9.*` schema changes. Currently DDL is run manually via `infrastructure/db.py`. Low risk in the current single-tenant dev setup.